==========

PROJECT REPORT: ERP Clone

==========

SUBMITTED BY

- Pranjal Nath Goswami
- Rahul Singh Samant
- Ravi Singh

SUBMITTED TO

Graphic Era Hill University, Bhimtal Campus

In partial fulfilment of the requirements for the

Bachelor of Computer Applications (BCA), 5th Semester

==========

1. ABSTRACT

==========

A web-based Student ERP Portal for Graphic Era Hill University that enables secure student onboarding, authentication, and profile access. The backend (Node.js/Express, TypeScript, MongoDB) handles registration with photo upload (Cloudinary), credential hashing, JWT cookie-based auth, and welcome email delivery (Nodemailer). The frontend (React, TypeScript, Vite, Tailwind) provides registration and login forms with validation, and a protected layout that fetches the student profile. Logout blacklists tokens to prevent reuse. The system improves onboarding speed, security of credentials, and centralizes student data access.

==========

2. PROBLEM STATEMENT

==========

Manual onboarding and profile sharing are slow and error-prone. Students need a single portal to register, receive credentials, and securely access their data with media storage and email notifications.

==========

3. OBJECTIVES

==========

- Secure authentication using JWT in httpOnly cookies.
- Streamlined registration with photo upload and automated welcome email.
- Authenticated profile retrieval with formatted student data.
- Safe logout via token blacklist.

==========

4. SCOPE

==========

In scope: Registration, login, protected profile fetch, logout, welcome email, image upload.

Out of scope (current): Fees payments, attendance, grading, timetables, role-based admin/teacher views.

==========

5. SYSTEM OVERVIEW

==========

Users: Students (primary).

Frontend: React (Vite, TS) at `http://localhost:5173`.

Backend: Express API at `http://localhost:3000`.

Data: MongoDB for users; Cloudinary for images; Nodemailer (Gmail) for email.

Auth: JWT (1h) in httpOnly cookie; token blacklist on logout.

==========

6. ARCHITECTURE (TEXTUAL)

==========

Client → API: Axios requests with `withCredentials` for protected routes.

API → DB: Mongoose models (`user`, `tokens` blacklist).

API → Cloudinary: Image uploads via Multer temp storage.

API → Gmail via Nodemailer: Welcome email.

Security: CORS allow `http://localhost:5173`; cookies `httpOnly`, `sameSite:lax`, `secure:false` (dev; use `secure:true`, `sameSite:'none'` in prod with HTTPS).

==========

7. TECHNOLOGY STACK

==========

Frontend: React, TypeScript, Vite, TailwindCSS, React Router, React Hook Form + Zod, Axios.

Backend: Node.js, Express, TypeScript, Mongoose, Multer, Cloudinary SDK, Nodemailer, JWT, Bcrypt, Cookie-Parser, CORS.

Database: MongoDB.

==========

8. FUNCTIONAL REQUIREMENTS (CURRENT)

==========

- Register student with photo upload (multipart form).
- Prevent duplicate email; hash passwords.
- Generate unique student ID (`ST28####`) and college email.
- Send welcome email with credentials.
- Login with userId + password → set JWT cookie.
- Fetch profile (protected route).
- Logout and blacklist token.

==========

9. NON-FUNCTIONAL REQUIREMENTS

==========

- Security: Bcrypt-hashed passwords; JWT httpOnly cookies; CORS restriction.

- Reliability: Graceful handling of missing files/emails.

- Usability: Simple forms, client-side validation.

- Performance: Lightweight endpoints; media offloaded to Cloudinary.

- Maintainability: Layered controllers/routes/middleware.

==========

10. NON-FUNCTIONAL REQUIREMENTS

==========

Auth & User Endpoints

- POST `/api/user/create`: Multer image → Cloudinary upload → bcrypt hash → welcome email.

- POST `/api/user/login`: Verify creds, issue JWT cookie.

- GET `/api/user/fetch`: Auth middleware verifies JWT, returns profile.

- GET `/api/user/logout`: Blacklist token, clear cookie.

Utilities

- `UserIdGenerator`, `formatDOB`, `genrateCollegeEmail`, `emailMsg` template.

Models

- `user`: name, email(unique), password(hashed), DOB, image, fees(default 0), userId, branch, phone, currentSem, course, collegeEmail.

- `tokens` (blacklist): token, expiry (TTL ~24h).

Middleware

- `auth`: JWT verify from cookie/Authorization.

- `multer`: disk storage for `image` field.

Frontend Flows

- Register page: validated form with image preview, posts multipart to `/create`.

- SignIn page: posts to `/login`, redirects to `/`.

- App layout: on mount, GET `/fetch` with credentials; redirect to `/sign-in` if unauthorized.

==========

11. DATA MODEL (SIMPLIFIED)

==========

- `user` collection: `{ name, email, password, DOB, image, fees, userId, branch, phone, currentSem, course, collegeEmail }`

- `tokens` collection: `{ token, expiry }` for blacklisting JWTs.

==========

12. SECURITY CONSIDERATIONS

==========

- Password hashing with bcrypt (pre-save hook).

- JWT in `httpOnly` cookie; `sameSite:lax`; `secure:false` (dev). For prod: `secure:true`, `sameSite:'none'`, HTTPS.

- Token blacklist on logout.

- CORS limited to frontend origin; adjust for prod domains.

- Recommend adding server-side validation (e.g., Joi/Zod) for defense in depth.

==========

13. TESTING SUMMARY

==========

Backend integration (manual/Postman):

- Register with/without image; duplicate email rejection.
- Login valid/invalid credentials.
- Fetch profile with/without valid token.
- Logout then verify fetch fails (blacklisted token).

Frontend checks:

- Form validation messages.
- Redirect to `/sign-in` when unauthorized.
- Successful login redirects to `/`.

==========

14. DEPLOYMENT NOTES

==========

Backend env vars: `PORT`, `MONGO_URL`, `JWT_SECRET`, `CLOUDINARY_*`, `USER`, `PASS`, `CORS_ORIGIN`.

Frontend: `npm run build`; host static (Netlify/Vercel).

Backend: `tsc` + `node dist` or `ts-node` (dev); host on Render/EC2/Fly.

- DB: MongoDB Atlas; Media: Cloudinary; Email: Gmail App Password.

==========

15. RESULTS

==========

- Student registration with photo and welcome email works.
- Secure login sets JWT httpOnly cookie.
- Protected profile fetch returns correct data.
- Logout invalidates session via blacklist.

==========

16. LIMITATIONS / FUTURE WORK

==========

- Add role-based access (admin/teacher).
- Add attendance, grades, fees payment, timetable.
- Stronger server-side validation and rate limiting.
- HTTPS + secure cookies in production.
- Observability (logs/metrics) and admin dashboard.

==========

17. HOW TO RUN (DEV)

==========

Backend

```bash
cd Backend
npm install
npm run dev   # or ts-node src/server.ts
```

Frontend

```bash
cd Frontend
npm install
npm run dev
```

==========

18. REFERENCES

==========

- React, Express, MongoDB, Cloudinary, Nodemailer, JWT, bcrypt official docs.